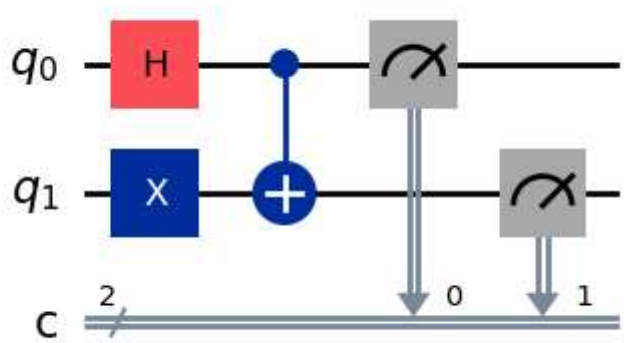
In [2]:
```
# Importing standard Qiskit libraries
from qiskit import QuantumCircuit,transpile
from qiskit.visualization import *
# qiskit-ibmq-provider has been deprecated.
# Please see the Migration Guides in https://ibm.biz/provider_migration_guide for m
from qiskit_ibm_runtime import QiskitRuntimeService, Sampler, Estimator, Session, O
# Loading your IBM Quantum account(s)
service = QiskitRuntimeService(channel="ibm_cloud")
from qiskit_aer import AerSimulator
from qiskit_ibm_runtime import QiskitRuntimeService
# Invoke a primitive. For more details see https://docs.quantum.ibm.com/run/primiti
# result = Sampler().run(circuits).result()
import numpy as np
# Importing standard Qiskit libraries
from qiskit_aer import Aer
#from qiskit import QuantumCircuit, transpile, assemble, IBMQ, execute
from qiskit.quantum_info import Statevector
from qiskit.visualization import plot_bloch_multivector, plot_histogram
```

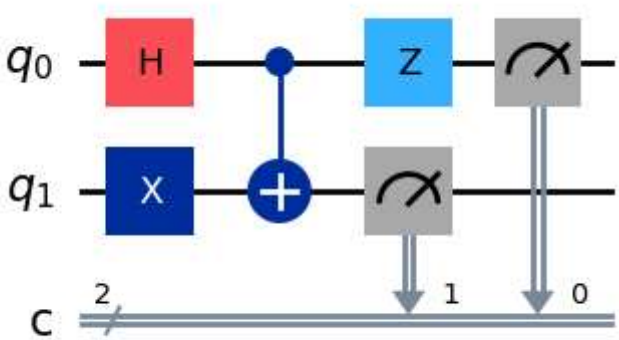
In [3]:
```
#|psi+>

# Create a Quantum Circuit with 2 qubits and 2 classical bits
qc = QuantumCircuit(2, 2)

# Step 1: Apply X gate on qubit 1 to flip it to |1>
qc.x(1)

# Step 2: Apply Hadamard gate on qubit 0 to create superposition
qc.h(0)

# Step 3: Apply CNOT gate with control qubit 0 and target qubit 1 to create entangl
qc.cx(0, 1)

# Step 4: Measure both qubits
qc.measure([0, 1], [0, 1])

# Draw the circuit
qc.draw('mpl')
```

Out[3]:



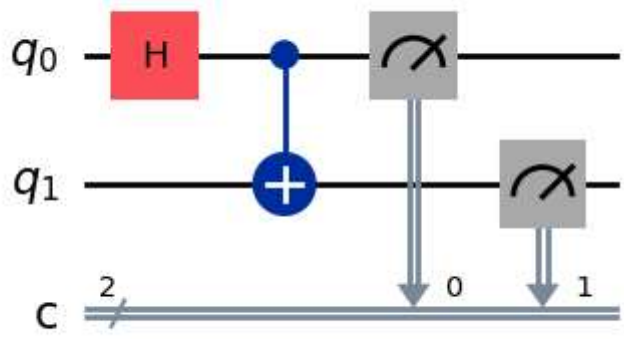
In [4]:
```
#|psi->

# Create a Quantum Circuit with 2 qubits and 2 classical bits
qc = QuantumCircuit(2, 2)

# Step 1: Apply X gate on qubit 1 to flip it to |1>
qc.x(1)

# Step 2: Apply Hadamard gate on qubit 0 to create superposition
qc.h(0)

# Step 3: Apply CNOT gate with control qubit 0 and target qubit 1 to create entangl
qc.cx(0, 1)

# Step 4: Apply Z gate on qubit 0 to introduce the phase needed for |psi->
qc.z(0)

# Step 5: Measure both qubits
qc.measure([0, 1], [0, 1])

# Draw the circuit
qc.draw('mpl')
```

Out[4]:



In [5]:
```
#|phi+>

# Create a Quantum Circuit with 2 qubits and 2 classical bits
qc = QuantumCircuit(2, 2)

# Step 1: Apply Hadamard gate on qubit 0 to create superposition
qc.h(0)

# Step 2: Apply CNOT gate with control qubit 0 and target qubit 1 to create entangl
qc.cx(0, 1)

# Step 3: Measure both qubits
qc.measure([0, 1], [0, 1])

# Draw the circuit
qc.draw('mpl')
```

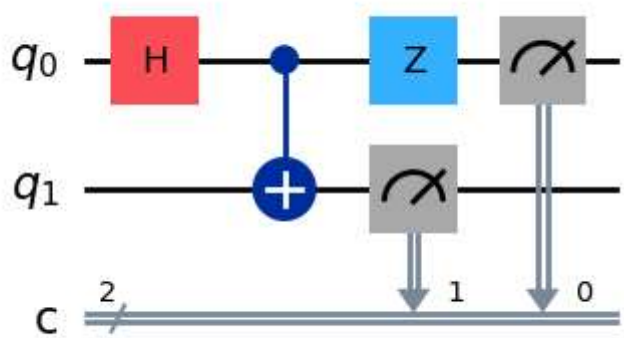
Out[5]:



In [6]:

```
#|phi->

qc_phi_minus = QuantumCircuit(2, 2)

# Step 1: Apply Hadamard on qubit 0
qc_phi_minus.h(0)

# Step 2: Apply CNOT with control qubit 0, target qubit 1
qc_phi_minus.cx(0, 1)

# Step 3: Apply Z gate on qubit 0 to introduce the phase (minus sign)
qc_phi_minus.z(0)

# Step 4: Measure both qubits
qc_phi_minus.measure([0, 1], [0, 1])

# Draw the circuit
qc_phi_minus.draw('mpl')
```

Out[6]:



In [2]:
```
import qiskit
```

In [3]:
```
qiskit.__version__
```

Out[3]:
```
'1.4.2'
```

In [4]:
```
token = "MK9bZuASn2QEw3VvroU7qg3AVjaRfq07O4U6qsJwPZ5F"
```

In [5]:
```
from qiskit_ibm_runtime import QiskitRuntimeService

service = QiskitRuntimeService.save_account(
    channel="ibm_cloud",    # or "ibm_quantum"
    token=token,            # IBM Cloud API key
    instance="instance1",       # <-- this should be your *CRN* or instance name
    name="default",      # optional
    set_as_default=True,
    overwrite=True
)
```

In [6]:
```
service = QiskitRuntimeService()
print(service.backends())
```

```
[<IBMBackend('ibm_brisbane')>, <IBMBackend('ibm_torino')>]
```

In [7]:
```
service.backends(simulator=False, operational=True, min_num_qubits=5)
```

Out[7]:
```
[<IBMBackend('ibm_brisbane')>, <IBMBackend('ibm_torino')>]
```

In [8]:
```
service.least_busy(operational=True, min_num_qubits=5)
```

Out[8]:
```
<IBMBackend('ibm_torino')>
```

In [9]:
```
backend = service.backend("ibm_brisbane")

print(
    f"Name: {backend.name}\n"
    f"Version: {backend.version}\n"
    f"No. of qubits: {backend.num_qubits}\n"
)
```

```
Name: ibm_brisbane
Version: 2
No. of qubits: 127
```

In [10]:
```
from qiskit import QuantumCircuit
from qiskit_ibm_runtime import QiskitRuntimeService, SamplerV2 as Sampler

# Create empty circuit
example_circuit = QuantumCircuit(2)
example_circuit.measure_all()

# You'll need to specify the credentials when initializing QiskitRuntimeService, i
```

```
service = QiskitRuntimeService()
backend = service.backend("ibm_brisbane")
sampler = Sampler(backend)
job = sampler.run([example_circuit])
print(f"job id: {job.job_id()}")
result = job.result()
print(result)
```

```
job id: d3lqemb4kkus739d1640
PrimitiveResult([SamplerPubResult(data=DataBin(meas=BitArray(<shape=(), num_shots=40
96, num_bits=2>)), metadata={'circuit_metadata': {}})], metadata={'execution': {'exe
cution_spans': {'__type__': 'ExecutionSpans', '__value__': {'spans': [DoubleSliceSpa
n(<start='2025-10-12 13:04:27', stop='2025-10-12 13:04:29', size=4096>)]}}}, 'versio
n': 2})
```

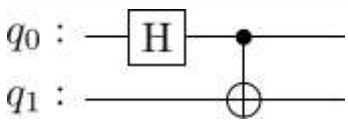
In [11]:

```
from qiskit import QuantumCircuit
from qiskit_ibm_runtime import QiskitRuntimeService, SamplerV2 as Sampler

# Create empty circuit
example_circuit = QuantumCircuit(2)
example_circuit.measure_all()

# You'll need to specify the credentials when initializing QiskitRuntimeService, if
service = QiskitRuntimeService()
backend = service.least_busy(operational=True, simulator=False)

sampler = Sampler(backend)
job = sampler.run([example_circuit])
print(f"job id: {job.job_id()}")
result = job.result()
print(result)
```

```
job id: d3lqepj4kkus739d1670
PrimitiveResult([SamplerPubResult(data=DataBin(meas=BitArray(<shape=(), num_shots=40
96, num_bits=2>)), metadata={'circuit_metadata': {}})], metadata={'execution': {'exe
cution_spans': {'__type__': 'ExecutionSpans', '__value__': {'spans': [DoubleSliceSpa
n(<start='2025-10-12 13:04:40', stop='2025-10-12 13:04:42', size=4096>)]}}}, 'versio
n': 2})
```

In [12]:
```
from qiskit import QuantumCircuit
from qiskit.quantum_info import SparsePauliOp
from qiskit.transpiler.preset_passmanagers import generate_preset_pass_manager
from qiskit_ibm_runtime import EstimatorV2 as Estimator

# Create a new circuit with two qubits
qc = QuantumCircuit(2)

# Add a Hadamard gate to qubit 0
qc.h(0)

# Perform a controlled-X gate on qubit 1, controlled by qubit 0
qc.cx(0, 1)

# Return a drawing of the circuit using MatPlotLib ("mpl"). This is the
# last line of the cell, so the drawing appears in the cell output.
# Remove the "mpl" argument to get a text drawing.
qc.draw("latex")
```

Out[12]:

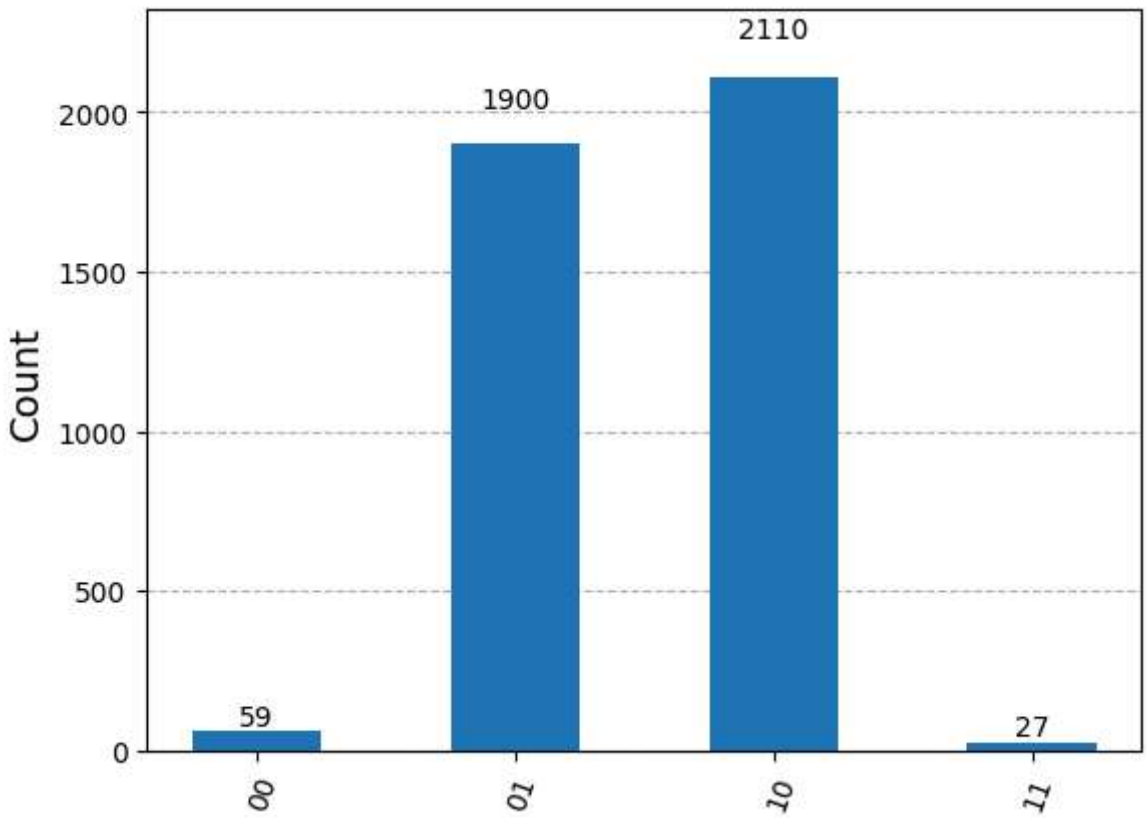


In [13]:
```
#Simulate and measure bell state on available device

from qiskit import transpile
from qiskit_ibm_runtime import Session
from qiskit.visualization import plot_histogram
import matplotlib.pyplot as plt
```

In [14]:
```
#|psi+>
qc = QuantumCircuit(2, 2)
qc.x(1)
qc.h(0)
qc.cx(0, 1)
qc.measure([0, 1], [0, 1])

backend = service.least_busy(simulator=False)
qc_transpiled = transpile(qc, backend=backend)

sampler = Sampler(backend)
job = sampler.run([qc_transpiled])
result = job.result()

counts = result[0].data.c.get_counts()
print("Measurement results:", counts)

plot_histogram(counts)
```

```
Measurement results: {'01': 1900, '10': 2110, '00': 59, '11': 27}
```

Out[14]:

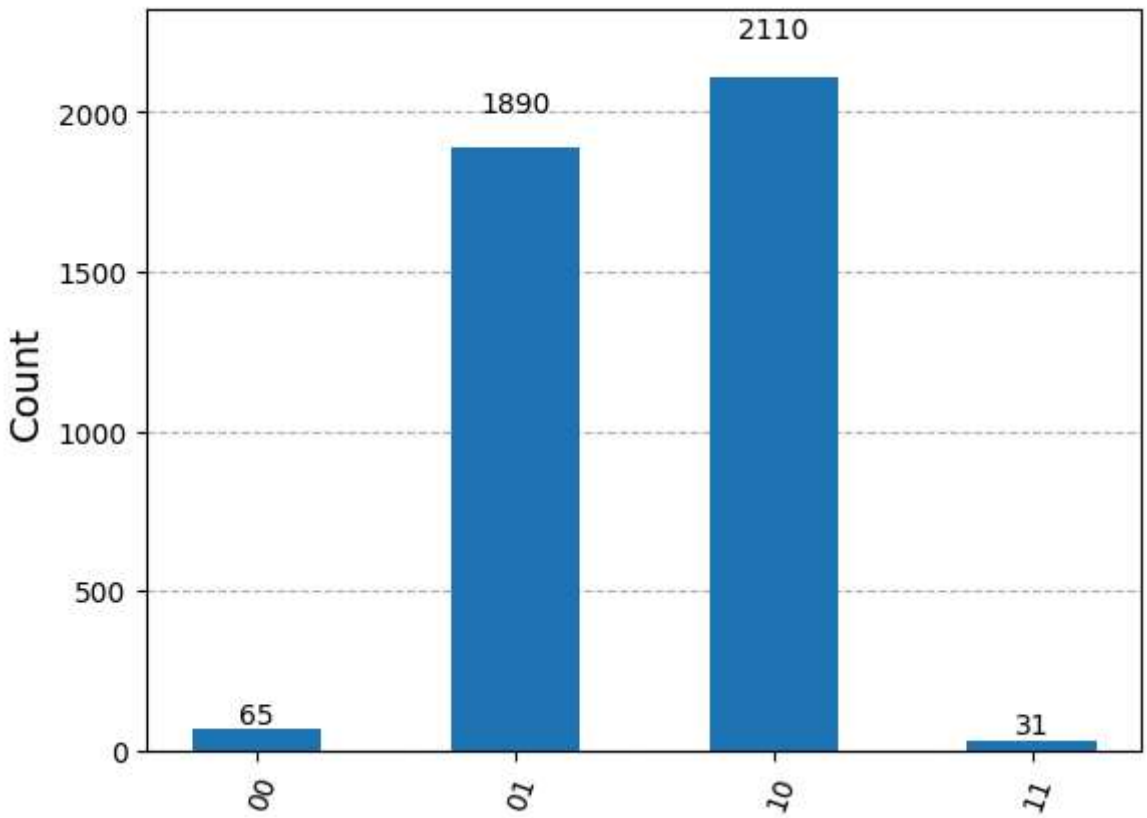


In [15]:

```
#|psi->
qc = QuantumCircuit(2, 2)
qc.x(1)
qc.h(0)
qc.cx(0, 1)
qc.z(0)
qc.measure([0, 1], [0, 1])

backend = service.least_busy(simulator=False)
qc_transpiled = transpile(qc, backend=backend)

sampler = Sampler(backend)
job = sampler.run([qc_transpiled])
result = job.result()

counts = result[0].data.c.get_counts()
print("Measurement results:", counts)

plot_histogram(counts)
```

```
Measurement results: {'10': 2110, '01': 1890, '11': 31, '00': 65}
```

Out[15]:

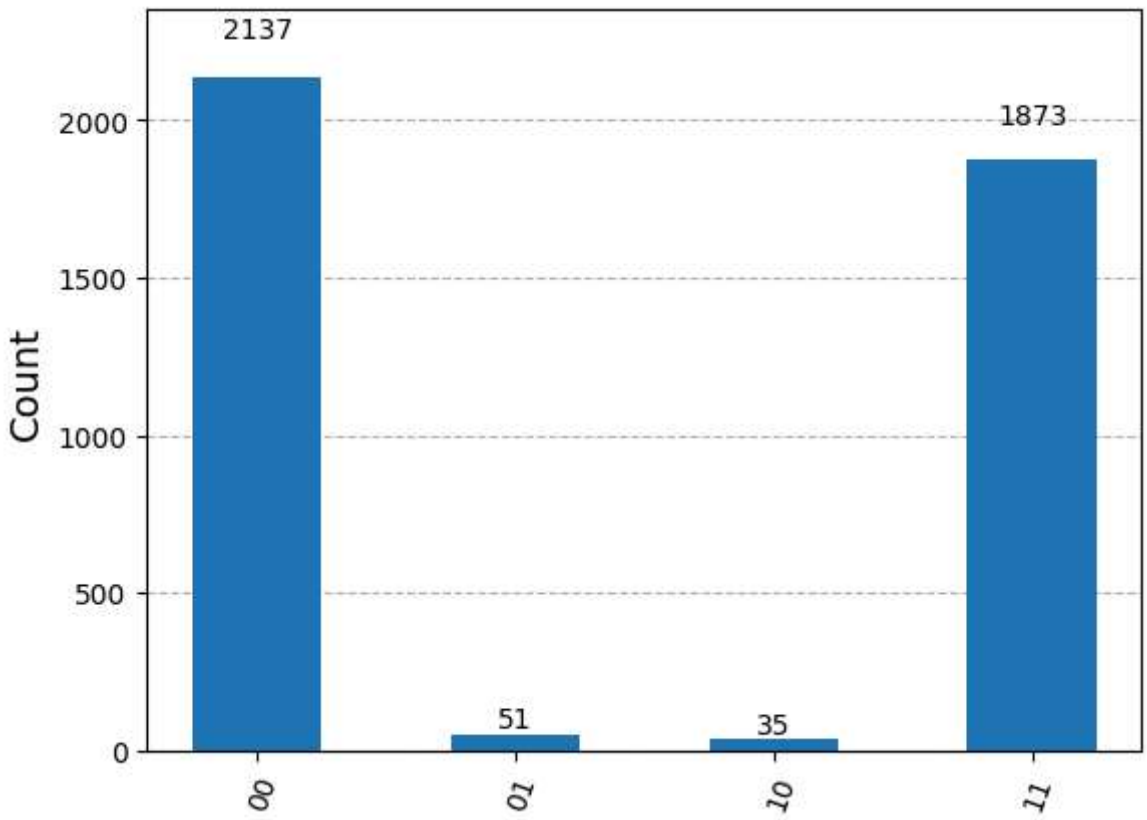


In [16]:

```
#|phi+>
qc = QuantumCircuit(2, 2)
qc.h(0)
qc.cx(0, 1)
qc.measure([0, 1], [0, 1])

backend = service.least_busy(simulator=False)
qc_transpiled = transpile(qc, backend=backend)

sampler = Sampler(backend)
job = sampler.run([qc_transpiled])
result = job.result()

counts = result[0].data.c.get_counts()
print("Measurement results:", counts)

plot_histogram(counts)
```

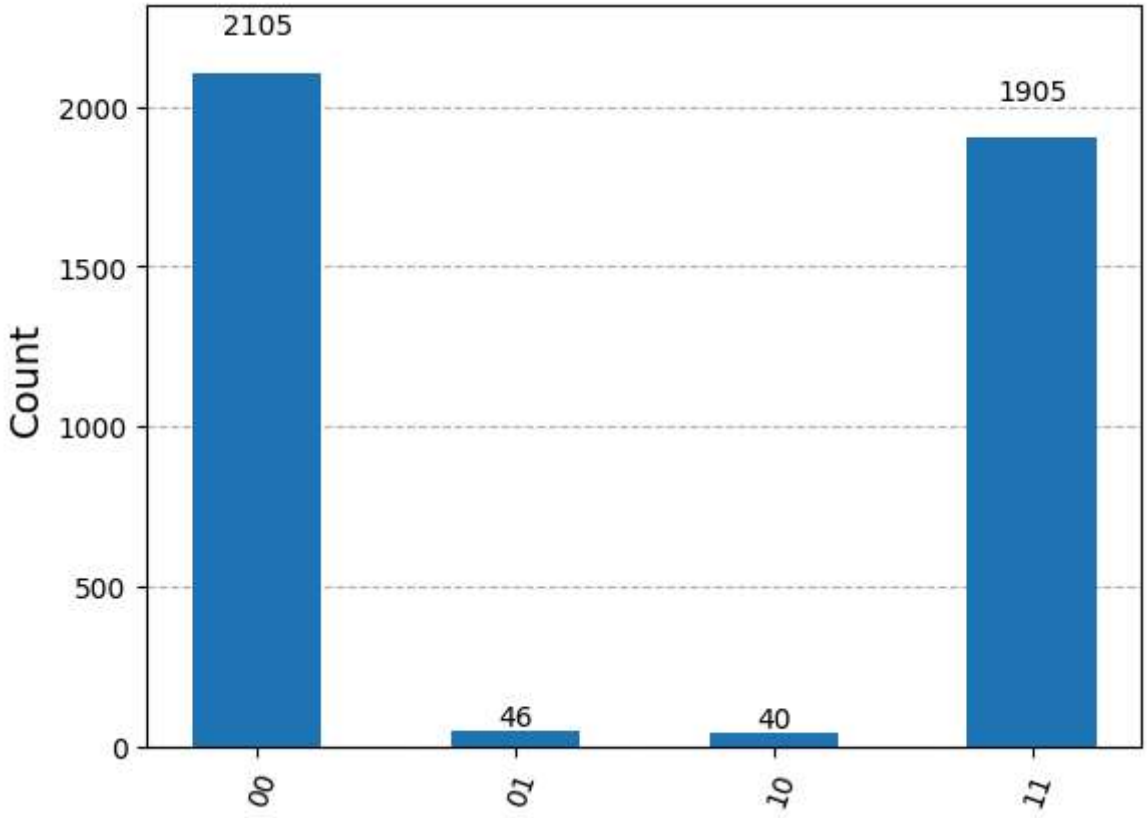
```
Measurement results: {'11': 1873, '00': 2137, '01': 51, '10': 35}
```

Out[16]:



In [17]:

```
#|phi->
qc = QuantumCircuit(2, 2)
qc.h(0)
qc.cx(0, 1)
qc.z(0)
qc.measure([0, 1], [0, 1])

backend = service.least_busy(simulator=False)
qc_transpiled = transpile(qc, backend=backend)

sampler = Sampler(backend)
job = sampler.run([qc_transpiled])
result = job.result()

counts = result[0].data.c.get_counts()
print("Measurement results:", counts)

plot_histogram(counts)
```

```
Measurement results: {'11': 1905, '00': 2105, '10': 40, '01': 46}
```

Out[17]: